GH-300 · GITHUB COPILOT · VISUAL CHEAT SHEET

GitHub Copilot, at a glance

Six domains · pass mark 700 · the developer stays accountable

L1 RESPONSIBLE AI · L2 PLANS · L3 IDE & CHAT · L4 AGENTS & MCP · L5 CLI · L6 GITHUB.COM · L7 CUSTOMISE · L8 ADMIN · L9 DATA FLOW · L10 PROMPTING · L11 PRODUCTIVITY · L12 EXCLUSIONS · L13 SAFEGUARDS

15–20%	25–30%	10–15%	10–15%	10–15%	10–15%
Responsible AI	Copilot features	Data & architecture	Prompting	Productivity	Privacy & safeguards

PART A · USE COPILOT RESPONSIBLY

L1 Responsible AI

DOMAIN 1 · 15–20%

Hallucination
Confident but wrong: invented APIs, packages, parameters.

Insecure patterns
Echoes vulnerable code seen in training data.

Bias
Skewed names, examples, assumptions about users.

Public code match
Rare verbatim matches raise licence questions.

VALIDATE BEFORE YOU TRUST

Copilot suggests a new login handler. What happens next?

1. **Read** — understand every line
2. **Test** — unit + edge cases
3. **Scan** — code, secrets, deps
4. **Human review** — PR approval ✓

TAKEAWAY
Copilot proposes, the developer decides. Accountability never moves to the tool.

MICROSOFT'S 6 PRINCIPLES

- Fairness
- Reliability & safety
- Privacy & security
- Inclusiveness
- Transparency
- Accountability

ALWAYS TRUE

- ✓ Human review and tests before merge
- ✓ Code scanning, secret scanning, Dependabot still apply
- ✓ Report bad output with 👍 / 👎 feedback

✗ "GitHub tested it, merge it"
✗ "Copilot knows the business rules"
✗ "Public code filter makes code secure"

L2 Plans & surfaces

WHO GETS WHAT

FROM PASSIVE HELP TO AUTONOMY

1. **Suggestions** — inline + next edit suggestions
2. **Chat** — ask, explain, edit; you drive
3. **Agents** — agent mode, Copilot CLI
4. **Cloud agent** — issue → pull request

WHICH PLAN?		
Trying Copilot, individual	→	**Free · 2,000 completions/mo, AI credit allowance, select models**
Individual developer, daily use	→	**Pro · unlimited completions + monthly AI credits**
Heavy individual use, premium models	→	**Pro+ / Max**
Organisation needs policies + content exclusions	→	**Business**
Enterprise-wide policies on GitHub Enterprise Cloud	→	**Enterprise**

OLD WORDING

Premium requests

Model multipliers per request. Still appears in older questions and courses.

SINCE 1 JUNE 2026

GitHub AI Credits

1 credit = $0.01. Same idea, new unit. Completions and NES aren't billed on paid plans.

TAKEAWAY

Content exclusions, org policies, audit log and IP indemnity are Business / Enterprise features. "Coding agent" is now the **cloud agent**.

PART B · COPILOT FEATURES (25–30%)

L3 IDE & Chat

SUGGESTIONS, MODES, COMMANDS

Inline

Ghost text as you type or from a comment.

Next edit (NES)

Predicts where you'll edit next and offers it.

Inline chat

Ctrl/⌘+I: prompt a change in place.

Chat view

Conversation, model picker, @ / # context.

BUILT-IN AGENTS (FORMERLY "CHAT MODES")

A **Ask**
questions & explanations; no file changes

›

E **Edit · legacy**
Copilot Edits: you pick files. Deprecated in VS Code 1.110, folded into Agent

›

P **Plan**
research + plan, then hand off

›

G **Agent**
finds files, edits, runs tools & tests

@ PARTICIPANTS

`@workspace`	**whole project**
`@vscode`	**editor features**
`@terminal`	**shell output**
`@github`	**repos, issues, PRs**

/ SLASH COMMANDS

`/explain`	**explain code**
`/fix`	**fix problems**
`/tests`	**write tests**
`/doc`	**doc comments**
`/new` · `/clear`	**scaffold · reset**

CONTEXT VARIABLES

`#file`	**a file**
`#selection`	**highlighted code**
`#codebase`	**search repo**
`#editor`	**visible code**
`#terminalSelection`	**terminal text**

KEYBOARD (VS CODE)		
Accept · dismiss	→	**Tab · Esc**
Next / previous suggestion	→	**Alt/⌥ +] · Alt/⌥ + [**
Accept next word	→	**Ctrl/⌘ + →**
Inline chat · Chat view	→	**Ctrl/⌘ + I · Ctrl+Alt+I (⌃⌘I)**

TAKEAWAY

VS Code now ships **Ask, Plan and Agent**; Edit mode is deprecated but "Copilot Edits" is still in the study guide, so know what it was. Chat only knows its context window: attach files with #, new thread per topic.

L4 Agent mode, MCP & sessions

MULTI-STEP WORK

THE AGENT LOOP

"Add pagination to the orders API and make the tests pass."

1. **Gather** — search & read files
2. **Edit** — across the project
3. **Run tools** — build, tests, MCP · you approve
4. **Self-correct** — until green, then you review

MCP

Model Context Protocol: adds tools & data (GitHub, DBs, APIs). Config e.g. `.vscode/mcp.json`; org **MCP policy** can block.

Agent sessions

Each task is a session: view, resume, stop. Local and cloud sessions side by side.

Subagents

Delegated task in its **own context window**; returns only the result.

.vscode/mcp.json

```
{ "servers": {
    "github":   { "type": "http",  "url": "https://api.githubcopilot.com/mcp/" },
    "postgres": { "type": "stdio", "command": "npx", "args": ["-y", "@modelcontextprotocol/server-
postgres"] }
} }
```

WHICH TOOL FOR THE TASK?

Quick question, explain code	→	**Ask mode**
Change known files, review each edit	→	**Copilot Edits (legacy) · now Agent with diff review**
Unknown scope, build/test loop	→	**Agent mode**
Agent must reach a database or API	→	**Add an MCP server**
Main chat bloated by research	→	**Subagent**
Long task, don't need to watch	→	**Cloud agent**

L5 GitHub Copilot CLI

AN AGENT IN THE TERMINAL

INSTALL → SIGN IN → WORK

1. **Install** — npm · brew --cask · winget · script
2. **/login** — needs a Copilot plan
3. **Trust folder** — run `copilot` in project
4. **Approve** — file writes & commands

Copilot CLI

```
npm install -g @github/copilot          # Node 22+
brew install --cask copilot-cli          # macOS / Linux · Windows: winget install GitHub.Copilot
(PowerShell 6+)
copilot                                  # interactive session
copilot -p "Summarise open PRs" --allow-tool 'shell(git)'   # scripted
/login /model /delegate /mcp /agent /usage /clear    # in-session · @file to reference · !cmd runs shell
```

REMEMBER

- ✓ `/delegate` hands the task to the cloud agent → PR
- ✓ Org / enterprise policy can disable the CLI
- ✓ Replaces the old `gh copilot` extension
- ✓ Supports MCP, AGENTS.md, custom agents, skills, hooks
- ✓ PAT auth: fine-grained token with **Copilot Requests** permission

WATCH OUT

- ✗ Content exclusions: "not applicable" to the CLI
- ✗ Pre-approving all tools in scripts removes the safety prompt
- ✗ Each prompt uses AI credits

L6 Copilot on GitHub.com

WHERE TEAMS COLLABORATE

CLOUD AGENT (FORMERLY CODING AGENT)

A backlog issue: "Add CSV export to the reports page."

1. **Assign** — issue → Copilot, or `/delegate`
2. **Sandbox** — runs in GitHub Actions
3. **copilot/* branch** — plans, edits, tests
4. **Draft PR** — summary + session log
5. **Human review** — iterate via @copilot

Code review

Copilot as a PR reviewer. Standards from instruction files. Auto-review via **rulesets**.

PR summary

Description from the diff. Edit before posting.

Spaces

Curated, shareable context: repos, files, issues, notes.

Spark

Build and deploy small web apps from natural language.

TAKEAWAY

The cloud agent pushes only to its own branch. Branch protection, required reviews and workflow approvals still apply.

L7 Customisation files

TEAM KNOWLEDGE IN THE REPO

WHICH FILE?		
Repo-wide standards, applied automatically	→	`.github/copilot-instructions.md`
Rules only for some paths (e.g. *.ts)	→	`.github/instructions/*.instructions.md` + `applyTo`
Instructions shared by every agent	→	`AGENTS.md`
Reusable prompt run on demand (IDE only)	→	`.github/prompts/*.prompt.md` → `/name`
Persona with its own tools & model	→	**Custom agent in** `.github/agents/` **(e.g.** `reviewer.agent.md` **)**
Multi-step workflow loaded when relevant	→	**Agent skill:** `.github/skills/<name>/SKILL.md`
Command that must run at a lifecycle point	→	**Hook:** `.github/hooks/*.json`
Personal or org-wide defaults	→	**GitHub.com settings**

.github/instructions/api.instructions.md

```
---
applyTo: "src/api/**/*.py"
---
- Use FastAPI dependency injection.
- Every endpoint needs a pytest test.
```

INSTRUCTION FILE

Always applied

Loads automatically for chat, agents and review.

PROMPT FILE

On demand

You run it by name with `/name`. IDEs only, not GitHub.com or the CLI.

TAKEAWAY

Precedence: **personal > repository > organisation** (all combined). Instructions shape chat, agents and review, not plain completions.

L8 Organisation & enterprise admin

CENTRAL CONTROL

WHO DECIDES

1. **Enterprise** — sets policy, or "no policy"
2. **Organisation** — policies, seats, exclusions
3. **Repository** — exclusions, instruction files
4. **User** — IDE settings within limits

Policies

IDE, chat, CLI, cloud agent, MCP, previews, models, public code.

Seats

All members, teams or users. Remove inactive ones.

Audit log

`action:copilot` : seats, policy, exclusions. **No prompts.**

Metrics

Active users, acceptance, by IDE, language, model.

REST API · /orgs/{org}/copilot/...

```
gh api /orgs/acme/copilot/billing                    # seat breakdown + settings
gh api /orgs/acme/copilot/billing/seats              # seats + last_activity_at
gh api -X POST   /orgs/acme/copilot/billing/selected_users -f "selected_usernames[]=octocat"
gh api -X DELETE /orgs/acme/copilot/billing/selected_users -f "selected_usernames[]=octocat"
gh api /orgs/acme/copilot/metrics/reports/organization-28-day/latest   # usage metrics → download links
(NDJSON)
# the old GET /orgs/{org}/copilot/metrics was shut down on 2 Apr 2026
```

TAKEAWAY

Enterprise policy beats organisation policy. If the enterprise sets it, the org owner can't change it.

PART C · DATA, ARCHITECTURE AND PROMPTING

L9 Data flow & lifecycle

DOMAIN 3 · 10–15%

FROM KEYSTROKE TO SUGGESTION

1. **Type / prompt** — code, comment, chat
2. **Gather context** — cursor, file, open tabs, repo index
3. **Build prompt** — + instructions, history
4. **Proxy filters** — over TLS · toxicity, relevance, jailbreak
5. **LLM** — generates candidates
6. **Post-process** — security + public code checks
7. **Shown** — ghost text or chat
8. **Accept / reject** — feedback & telemetry

BEFORE THE MODEL

Proxy filtering

Blocks toxic, off-topic and prompt-injection attempts; applies policies and exclusions.

AFTER THE MODEL

Post-processing

Security filter (hard-coded creds, SQL injection) · **public code filter** (~150-char match) · **code referencing**.

DATA RETENTION & TRAINING		
Business / Enterprise	→	**Never used for training · IDE prompts not retained · other surfaces ~28 days**
Free / Pro / Pro+ / Max	→	**Used for training from 24 Apr 2026 unless the user opts out**
Private repository content at rest	→	**Not used for training**

Probabilistic
Different answers each run: review, regenerate.

Context window
Sees only part of the repo; long chats forget.

Knowledge cutoff
Suggests deprecated APIs: check docs.

L10 Prompt engineering

DOMAIN 4 · 10–15%

ANATOMY OF A STRONG PROMPT

Goal	Context	Constraints	Examples	Output
"Write a Python function that validates emails"	"for the signup form in #file:forms.py"	"standard library only, max 30 lines"	"a@b.co → True, a@b → False"	"plus three pytest tests"

Single
One clear task per prompt.

Specific
Explicit details and constraints.

Short
Concise, no padding.

Surround
Open files, good names, comments.

NO EXAMPLES

Zero-shot
"Write a function to parse ISO dates." Fine for common tasks.

1–3 EXAMPLES

Few-shot
Show input → output or a function to imitate. Best for your own formats.

TAKEAWAY

Chat sends your message + thread history + instruction files + attached context. New topic → new thread.

PART D · DEVELOPER PRODUCTIVITY

L11 Productivity, testing & security

DOMAIN 5 · 10–15%

Generate
Boilerplate, SQL, regex, scripts, configs.

Refactor
Small steps, tests green each time.

Document
`/doc`, READMEs, PR summaries.

Learn
`/explain` new code & languages; less context switching.

Sample data
Synthetic JSON, CSV, SQL. Never real PII.

Modernise legacy
Explain, lock behaviour with tests, translate.

SAFE LEGACY MODERNISATION

1. **Explain** — /explain the old module
2. **Tests first** — capture current behaviour
3. **Small steps** — refactor / translate
4. **Run & review** — tests + diff each step

ONE FUNCTION, MOCKS

Unit tests

`/tests` on a selection. Specify the framework (pytest, JUnit, Jest).

COMPONENTS TOGETHER

Integration tests

DB, API, services. Agent mode can write and run them.

EDGE CASES TO ASK FOR

empty / null · **boundaries** · **huge input** · **unicode** · **error paths** · **concurrency** · **invalid types**

Use meaningful **assertions**, and check a test can actually fail.

SECURITY & PERFORMANCE

- ✓ OWASP review, parameterised queries, validation
- ✓ Code scanning + **Copilot Autofix**, secret scanning, Dependabot
- ✓ Complexity, caching, N+1: benchmark before/after

PART E · PRIVACY, CONTENT EXCLUSIONS AND SAFEGUARDS

L12 Content exclusions

DOMAIN 6 · BUSINESS & ENTERPRISE ONLY

THREE LEVELS

R **Repository**
repo admins · applies to all Copilot users in that repo

O **Organisation**
any repo by URL, or `"*"` for all

E **Enterprise**
applies across all orgs

~30 minutes
to propagate · reload IDE

organisation content exclusion (YAML)

```yaml
"*":                          # every repository
  - "**/.env"
  - "**/secrets/**"
https://github.com/acme/payments:
  - "*.pem"
```

EXCLUDED FILE

What it blocks

No suggestions in it · not used as context · Ask chat can't read it · code review skips it.

KNOWN GAPS

Where it doesn't apply

Agent & Edit modes, **Copilot CLI**; third-party MCP servers; symlinks & remote filesystems; IDE may still pass type info. GitHub.com/Mobile support is in preview.

TAKEAWAY

Exclusions reduce exposure; they are not a vault. The real fix for secrets is to keep them out of the repo.

L13 Safeguards & troubleshooting

PUBLIC CODE · OWNERSHIP · FIXES

PUBLIC CODE: BLOCKED

Duplication detection filter

Suggestions matching ~150 characters of public GitHub code are dropped. Lowest licence risk.

PUBLIC CODE: ALLOWED

Code referencing

Matches are shown with their source and **licence** so you can decide.

Ownership

GitHub claims no ownership; you're responsible for what you accept.

IP indemnity

Business / Enterprise, conditions apply.

Who sets it

User on personal plans; enforced by org / enterprise policy.

READ THE SYMPTOM		
No suggestions anywhere	→	**Signed in? Seat? Language on? Network, proxy, outage?**
Stops in one file only	→	**Content exclusion · file too large · language off**
New exclusion not working	→	**Wait ~30 min / reload · path syntax · agent/CLI gap**
Suggestion vanishes mid-typing	→	**Public code or security filter**
Chat, CLI or MCP missing	→	**Org / enterprise policy · plan · preview off**
Usage paused	→	**AI credits or budget exhausted · rate limit**
Works at home, not in office	→	**HTTP proxy, custom certs, allowlist hosts**

✗ → ✓ Wrong-answer phrasebook

✗ Merge Copilot code because GitHub tested it	✓ Read, test, scan and review it yourself
✗ Content exclusions on Free or Pro	✓ Business and Enterprise only
✗ Exclusions stop agent mode and the CLI	✓ Known gap: agent mode, CLI, cloud agent
✗ Org owner overrides an enterprise policy	✓ Enterprise policy wins
✗ Audit log shows prompts and suggestions	✓ Only admin events (action:copilot)
✗ Business prompts train the model	✓ Never used for training
✗ Public code filter makes code secure	✓ It addresses licence risk only
✗ Prompt file loads automatically	✓ Instruction files are automatic; prompt files run by /name
✗ Use agent mode for a one-line question	✓ Ask mode
✗ Copilot Edits finds the files for you	✓ You picked the working set (legacy); Agent finds files
✗ Use GET /orgs/{org}/copilot/metrics	✓ Shut down Apr 2026: use the metrics reports endpoints
✗ Keep one giant chat thread	✓ New thread per topic
✗ Paste real customer data for samples	✓ Ask for synthetic data
✗ Premium requests and AI credits differ	✓ Same idea, renamed unit

↺ Quick recall

ACCOUNTABILITY
Always the developer

6 PRINCIPLES
Fairness · reliability · privacy · inclusiveness · transparency · accountability

BUILT-IN AGENTS
Ask · Plan · Agent (Edit = legacy Copilot Edits)

PARTICIPANTS
@workspace · @vscode · @terminal · @github

MCP
Tools & data for agents · `.vscode/mcp.json` · org MCP policy

CLI
`npm i -g @github/copilot` · `/delegate` → cloud agent

INSTRUCTIONS
`copilot-instructions.md` · `*.instructions.md` + applyTo · AGENTS.md

PROMPT / AGENTS / SKILLS / HOOKS
`*.prompt.md` (IDE) · `.github/agents/` · `SKILL.md` · `.github/hooks/*.json`

POLICY ORDER
Enterprise › organisation › user

SEATS API
`/orgs/{org}/copilot/billing/selected_users`

FILTERS
Proxy before model · security + public code after

4 S'S
Single · Specific · Short · Surround

EXCLUSION LEVELS
Repo · org · enterprise · ~30 min

PUBLIC CODE
Blocked = filter · Allowed = code referencing

? Read the signal

"Developer wants to change one function in place"	→	**Inline chat (Ctrl/⌘+I)**
"Add a feature across unknown files, fix the tests"	→	**Agent mode**
"Agent needs the team's Postgres database"	→	**MCP server**
"Hand a backlog issue to Copilot, get a PR"	→	**Cloud agent**
"Every PR in the org needs an AI review"	→	**Code review via ruleset / policy**
"New hires need answers over three repos"	→	**Copilot Spaces**
"Team always uses pytest and our logger"	→	**copilot-instructions.md**
"Same scaffold prompt every week"	→	**Prompt file**
"Add 200 users from HR data"	→	**REST API selected_users**
"Who changed the public code policy?"	→	**Audit log**
"Keep secrets/** away from Copilot everywhere"	→	**Org exclusion with "*"**
"Legal fears licensed code"	→	**Public code filter / code referencing**
"Pro user doesn't want prompts used for training"	→	**Opt out in settings**

★ Where each domain shows up

MISSION 1 · RESPONSIBLE AI

Untested auth code

A junior wants to merge Copilot's login handler unreviewed.

→ Read, test, scan, human review.

MISSION 2 · FEATURES

Feature across the repo

Unknown files, failing tests, needs the DB.

→ Agent mode + MCP server; review the diff.

MISSION 3 · DATA

"Where did my code go?"

Security asks what leaves the IDE.

→ Context → proxy → LLM → post-processing; Business data not retained or trained on.

MISSION 4 · PROMPTING

Vague suggestions

Copilot ignores the helper module.

→ Open it, #file it, 4 S's, few-shot example.

MISSION 5 · PRODUCTIVITY

COBOL to Java

Translate a legacy module safely.

→ Explain, tests first, small steps, review.

MISSION 6 · PRIVACY

.env in every repo

Keep secrets away from Copilot org-wide.

→ Org exclusion "*"; know the agent/CLI gap.

Technical validation log

CHECKED AGAINST GITHUB DOCS · NOV 2026

CORRECTED IN THIS EDITION		
Chat modes listed as Ask · Edit · Plan · Agent	→	**Edit mode deprecated in VS Code 1.110 (hidden by default, removed from 1.126). Built-ins are Ask, Plan, Agent; modes are now called agents**
`brew install copilot-cli`	→	**`brew install --cask copilot-cli` ; Windows needs PowerShell 6+**
`GET /orgs/{org}/copilot/metrics`	→	**Shut down 2 Apr 2026; use `…/copilot/metrics/reports/organization-1-day` or `organization-28-day/latest`**
Content-exclusion gaps	→	**Added third-party MCP servers, symlinks, remote filesystems; GitHub.com/Mobile support is preview; CLI is "not applicable"**
Prompt files everywhere	→	**IDE only (VS Code, Visual Studio; preview elsewhere); not GitHub.com or CLI**
Customisation list	→	**Added agent skills (`SKILL.md`) and hooks (`.github/hooks/*.json`)**
Free plan "auto model only"	→	**Docs say "a selection of models" plus an AI credit allowance**

CONFIRMED AS ACCURATE		
AI Credits from 1 Jun 2026, 1 credit = $0.01; completions & NES unbilled on paid plans	✓	**GitHub Docs / blog**
Free = 2,000 completions/month	✓	**GitHub Docs (plans)**
Training on Free/Pro/Pro+ data from 24 Apr 2026 unless opted out; never Business/Enterprise	✓	**GitHub blog**
Business/Enterprise: IDE prompts not retained; other surfaces (GitHub.com, Mobile, CLI) 28 days; engagement data 2 yrs	✓	**Trust Center FAQ**
CLI: `npm install -g @github/copilot` (Node 22+), winget, install script, `/login`	✓	**GitHub Docs (install)**
Content exclusion: Business/Enterprise; repo admins, org owners, enterprise owners; blocks suggestions, context, chat, code review	✓	**GitHub Docs (content exclusion)**
Instruction files, path-specific `applyTo` , AGENTS.md, custom agents in `.github/agents/`	✓	**Customization cheat sheet**
Cloud agent = renamed coding agent; runs in an Actions sandbox; opens PRs	✓	**GitHub Docs**

HOW TO USE THIS

The exam was written against the study guide (skills as of 7 Aug 2026), so it may still use older names like **Copilot Edits**, **coding agent** or **premium requests**. Recognise both the old and the new wording.

Built from the Microsoft Learn GH-300 study guide (skills measured 7 Aug 2026) and GitHub Docs, late Oct 2026. Copilot changes fast: confirm details in GitHub Docs. Icons: Lucide (ISC). Not affiliated with GitHub or Microsoft.